

数据结构与算法

第1章 绪论

张丽淼

光电信息获取与防护技术全国重点实验室

zhanglm@ahu.edu.cn

2026年3月4日

1. 数据结构的课程内容

- ❖ 各种数据的逻辑结构描述。
- ❖ 各种数据的存储结构表示。
- ❖ 各种数据结构的运算定义。
- ❖ 设计实现运算的算法。
- ❖ 分析算法的效率。

基本数据组织和数据处理方法

2.为什么要学习数据结构

❖ 编程基础

❖ 计算机及相关专业考研考博课程

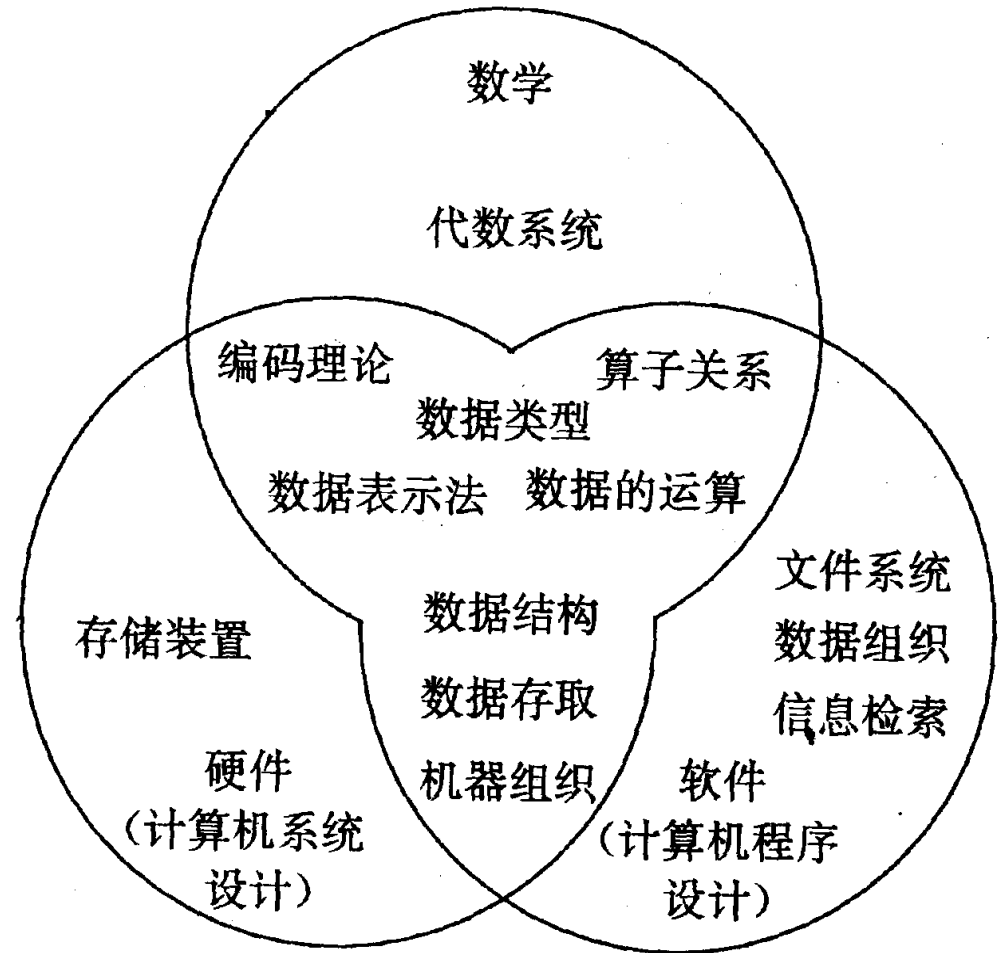
❖ 计算机等级考试课程

❖ 程序员考试课程



《数据结构》所处的地位：

介于数学、计算机硬件和计算机软件三者之间的一门核心课程



数据结构在计算机学科中的地位

Web信息处理

队列、图、字符、矩阵、哈希表、排序、索引、检索

人工智能

广义表、集合、有向图、搜索树

图形图像

队列、栈、图、矩阵、空间索引树、检索

数据库

线性表、多链表、排序、B+树

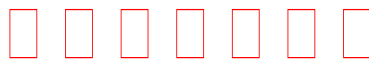
操作系统

队列、存储管理表、排序、目录树

编译原理

字符串、栈、哈希表、语法树

算法分析与设计



计算复杂性理论

程序设计语言

概率统计

计算概论

集合论与图论

3. 数据结构的学习目标

课程目的

- 能够分析研究计算机加工的对象特性，获得其**逻辑结构**，根据需求，选择合适**存储结构及其相应的算法**；
- 学习一些**常用的算法**；
- 复杂程序设计的训练过程，要求编写的程序**结构清楚和正确易读**；
- 初步掌握算法的**时间分析和空间分析技术**

同一求解问题通常有多种实现算法，通过时间复杂度与空间复杂度的分析，找出最好的实现算法。

例如，求 $1 + 2 + \dots n$ 。

算法1:

```
int fun1(int n)
{ int i, s=0;
  for (i=1; i<=n; i++)
    s+=i;
  return s;
}
```

算法2:

```
int fun2(int n)
{
  return (n+1)*n/2;
}
```



算法分析

算法2好于算法1

4. 课程学习指导

课程特点：内容抽象、概念性强、内容灵活、不易掌握

- 1.提前预习、认真听课、按时完成书面及上机作业
- 2.注意先修课程的知识准备
 - ✓ 离散数学、C语言
- 3.注意循序渐进：
 - ✓ 基本概念、基本思想、基本步骤、算法设计
- 4.注意培养算法设计的能力
 - ✓ 理解所讲算法、对此多做思考：若问题要求不同，应如何选择数据结构，设计有效的算法
 - ✓ “条条大路通罗马”

5. 考核方式

- 平时成绩：40%
 - 课后作业 20%
 - 课堂测试 20%
- 期中成绩：10%
- 期末成绩：50%

数据结构

还有比学习更快乐的事吗？
我看没有！



第1章 绪论



学习目标

1. 了解数据结构研究的主要内容
2. 掌握数据结构中涉及的基本概念
3. 掌握算法的时间、空间复杂度及其分析的简易方法

章节内容

1.1 什么是数据结构

1.2 基本概念和术语

1.3 抽象数据类型的表示和实现

1.4 算法和算法分析



1.1 什么是数据结构

📖 N.沃思 (Niklaus Wirth)教授提出:

程序=算法+数据结构

📖 电子计算机的主要用途:

☞ 早期:

主要用于数值计算——数学公式、方程

☞ 后来:

处理逐渐扩大到非数值计算领域, 能处理多种复杂的具有一定结构关系的数据——表、树、图等

书目自动检索系统

书目卡片

登录号:

书名:

作者名:

分类号:

出版单位:

出版时间:

价格:

书目自动检索系统

线性表

001	高等数学	樊映川	S01
002	理论力学	罗远祥	L01
003	高等数学	华罗庚	S01
004	线性代数	栾汝书	S02
.....			

书目文件

索引表

按书名

按作者名

按分类号

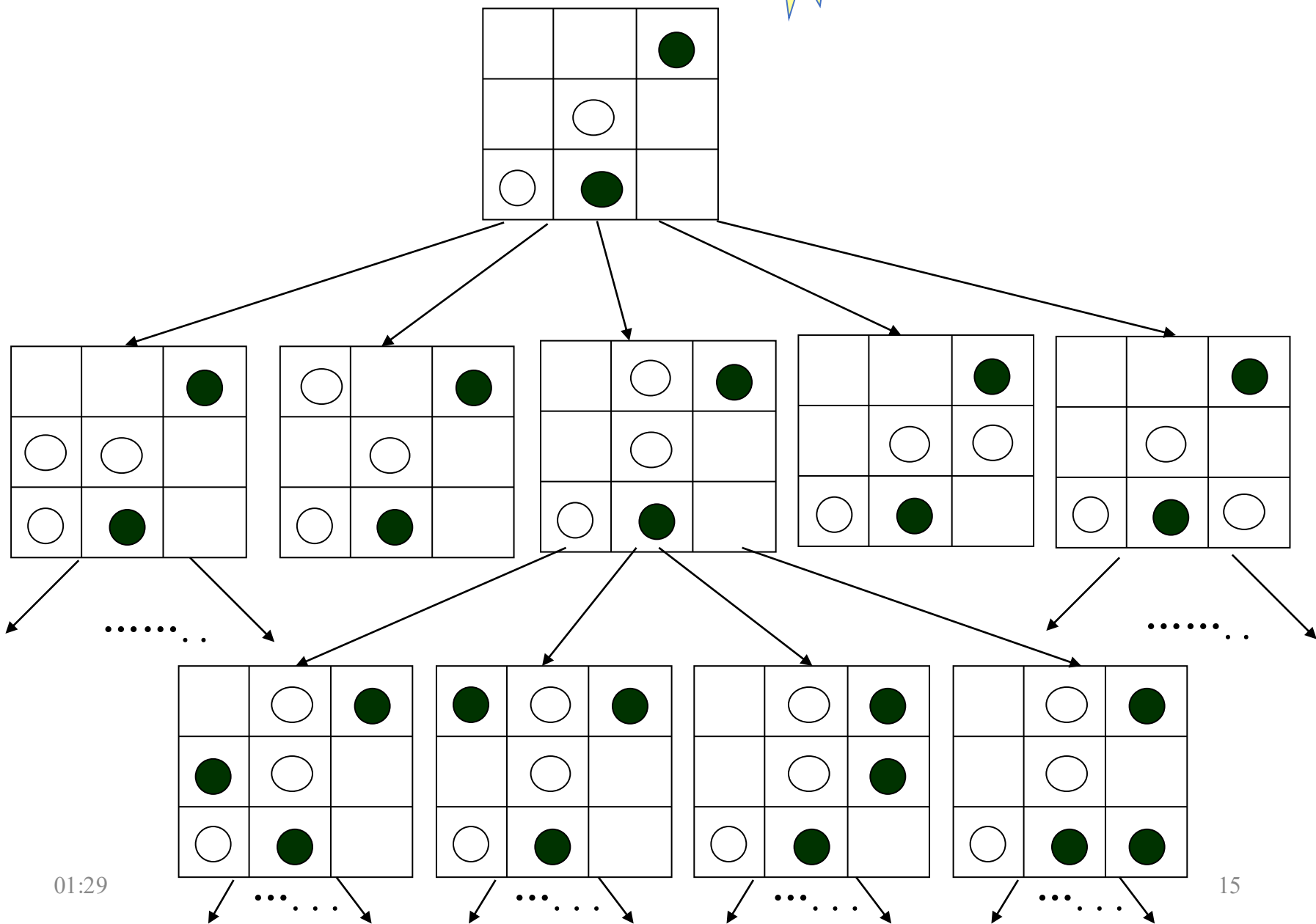
高等数学	001, 003.....
理论力学	002,
线性代数	004,
.....	

樊映川	001,...
华罗庚	002,...
栾汝书	004,...
.....	

L	002,...
S	001,003,
.....	

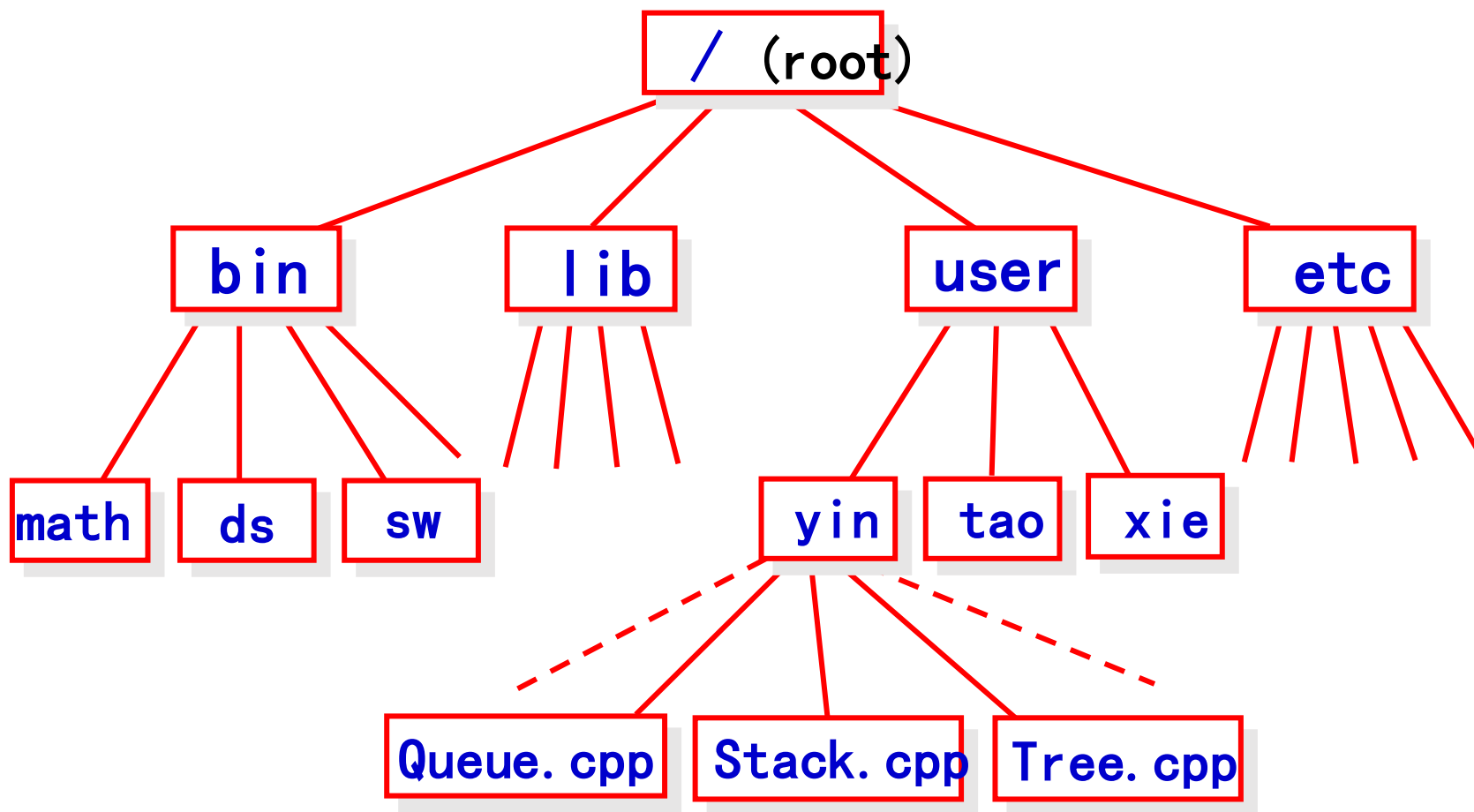
人机对弈问题

树

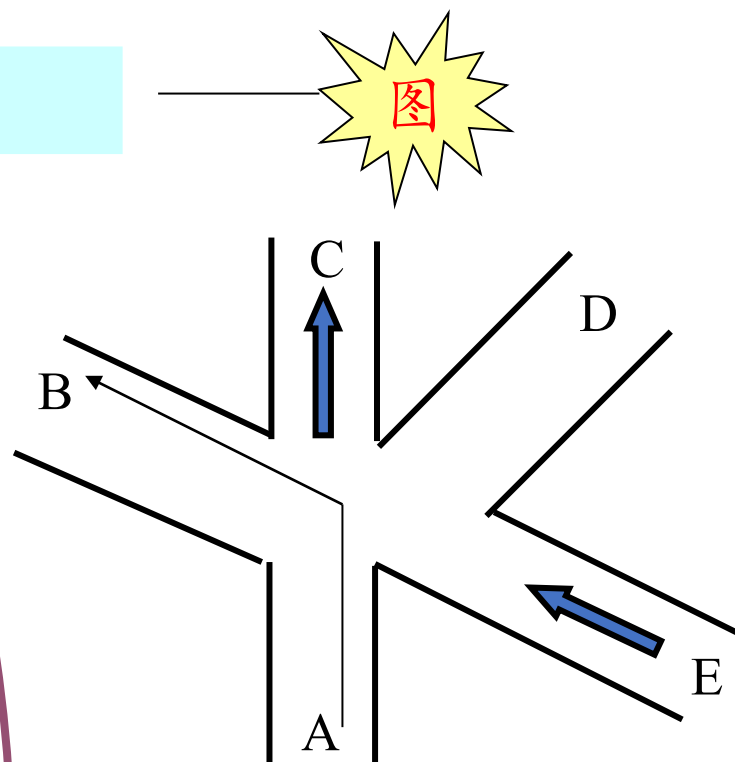
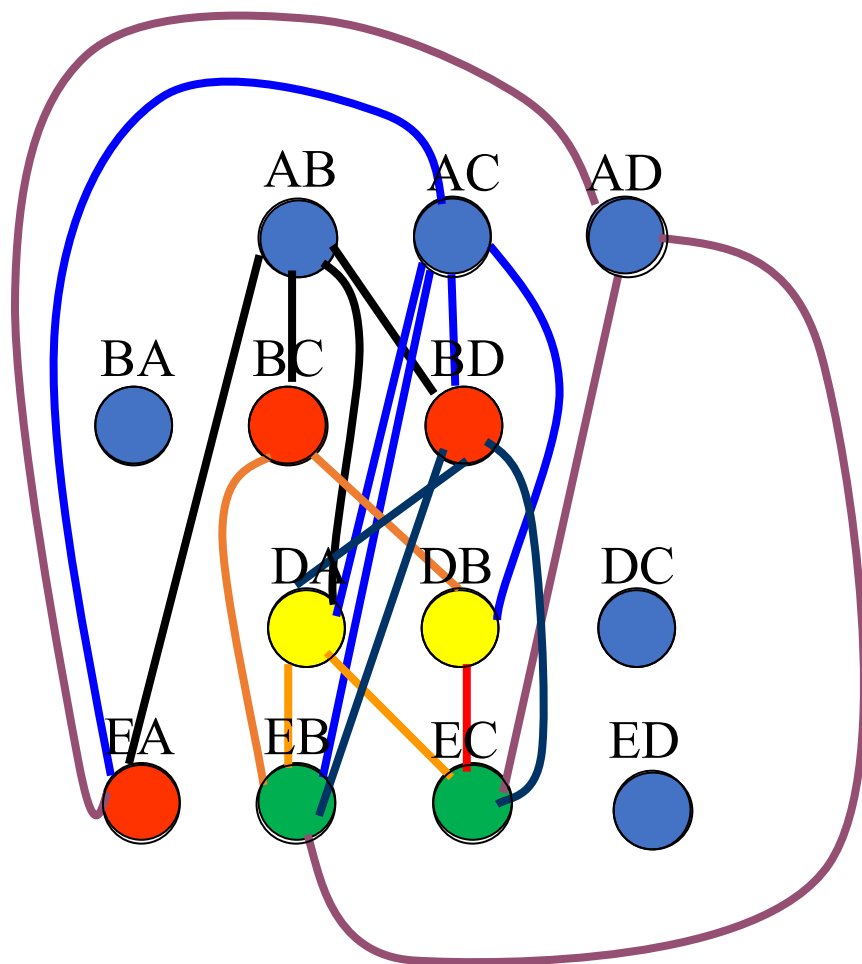


文件系统的系统结构图

树



多叉路口交通灯管理问题



顶点：一条通路

连线：不能同时通行

染色：有连线的两个顶点不能具有相同颜色

求解非数值计算的问题：

设计出合适的数据结构及相应的算法

即：首先要考虑对相关的各种信息如何表示、组织和存储？

数据结构的研究内容为：

研究非数值计算的程序设计问题中计算机的操作对象以及它们之间的关系和操作。



1.2 基本概念和术语

- 1、**数据** (data)—所有能输入到计算机中去的**描述客观事物的符号**
 - ◆ 数值性数据
 - ◆ 非数值性数据 (多媒体信息处理)
- 2、**数据元素** (data element) —数据的**基本单位**，也称**结点** (node) 或**记录** (record)
- 3、**数据项** (data item) —是构成数据元素的、有独立含义的数据**最小单位**，也称**域**(field)

三者之间的关系：数据 > 数据元素 > 数据项

例：学生表 > 个人记录 > 学号、姓名……

结构化数据示例

一个学生表

学号	姓名	性别	班号
1	张斌	男	9901
8	刘丽	女	9902
34	李英	女	9901
20	陈华	男	9902
12	王奇	男	9901
26	董强	男	9902
5	王萍	女	9901

数据项 (用于
描述数据元素
)

数据
元素

4、数据对象(Data Object): 相同特性数据元素的集合, 是数据的一个子集

- ◆ 整数数据对象

$$\mathbf{N} = \{ 0, \pm 1, \pm 2, \cdots \}$$

- ◆ 学生数据对象

- 学生基本信息表

默认情况下, 数据结构中讨论的数据都是数据对象。

5、数据结构（Data Structure）是相互之间存在一种或多种特定关系的数据元素的集合。

数据结构是带“结构”的数据元素的集合，“结构”就是指数据元素之间存在的关系。



数据结构两个层次：

逻辑结构

数据元素间抽象化的相互关系，与数据的存储无关，独立于计算机，它是从具体问题抽象出来的数学模型。

存储结构（物理结构）

数据元素及其关系在计算机存储器中的存储方式。

逻辑结构的表示

数据的逻辑结构是面向用户的，它有多种表示形式。

- 表示方法1——表格

学生表

学号	姓名	性别	班号
1	张斌	男	9901
8	刘丽	女	9902
34	李英	女	9901
20	陈华	男	9902
12	王奇	男	9901
26	董强	男	9902
5	王萍	女	9901

直接来源于现实世界

逻辑结构的表示

- 表示方法2——图形

在学生表中，用学号标识每个学生记录，其逻辑结构用图形表示如下：



逻辑结构的表示

● 表示方法3——二元组

二元组是一种通用的逻辑结构表示方法。

一个二元组表示为：

$$B=(D, R)$$

其中， B 是一种数据结构，它由数据元素的集合 D 和 D 上二元关系的集合 R 所组成。其中：

$D=\{d_i \mid 1 \leq i \leq n, n \geq 0\}$ ：数据元素的集合

$R=\{r_j \mid 1 \leq j \leq m, m \geq 0\}$ ：关系的集合

逻辑结构的表示

每个关系的用若干个序偶来表示：

- 序偶 $\langle x, y \rangle$ ($x, y \in D$) $\Rightarrow$ x 为第一元素, y 为第二元素。
- x 为 y 的 (直接) 前驱元素。
- y 为 x 的 (直接) 后继元素。
- 若某个元素没有前驱元素, 则称该元素为开始元素; 若某个元素没有后继元素, 则称该元素为终端元素。

序偶 $\langle x, y \rangle$ 表示 x 、 y 是有向的

序偶 (x, y) 表示 x 、 y 是无向的

学号	姓名	性别	班号
1	张斌	男	9901
8	刘丽	女	9902
34	李英	女	9901
20	陈华	男	9902
12	王奇	男	9901
26	董强	男	9902
5	王萍	女	9901



每个学生记录用学号标识

二元组逻辑表示:

$$B = (D, R)$$

$$D = \{ 1, 8, 34, 20, 12, 26, 5 \}$$

$$R = \{ r \}$$

$$r = \{ \langle 1, 8 \rangle, \langle 8, 34 \rangle, \langle 34, 20 \rangle, \langle 20, 12 \rangle, \langle 12, 26 \rangle, \langle 26, 5 \rangle \}$$

逻辑结构的类型

划分方法一

(1) 线性结构

有且仅有一个开始和一个终端结点，并且所有结点都最多只有一个直接前趋和一个后继。

例如：线性表、栈、队列、串

(2) 非线性结构

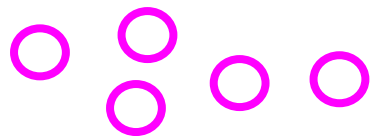
一个结点可能有多个直接前趋和直接后继。

例如：树（一对多）、图（多对多）

逻辑结构的类型

划分方法二

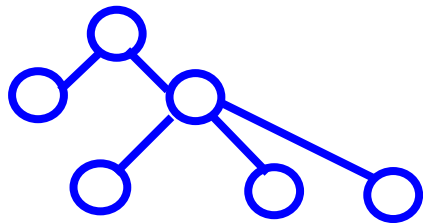
集合——数据元素间除“同属于一个集合”外，无其它关系



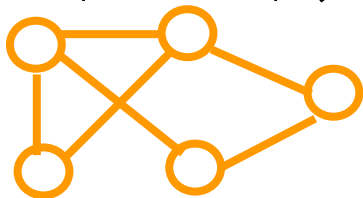
线性结构——一个对一个，如线性表、栈、队列



树形结构——一个对多个，如树



图形结构——多个对多个，如图



逻辑结构的类型

【例1.3】有一种数据结构 $B2 = (D, R)$ ，其中

$D = \{48, 25, 64, 57, 82, 36, 75\}$

$R = \{r_1, r_2\}$

$r_1 = \{ \langle 25, 36 \rangle, \langle 36, 48 \rangle, \langle 48, 57 \rangle, \langle 57, 64 \rangle, \langle 64, 75 \rangle, \langle 75, 82 \rangle \}$

$r_2 = \{ \langle 48, 25 \rangle, \langle 48, 64 \rangle, \langle 64, 57 \rangle, \langle 64, 82 \rangle, \langle 25, 36 \rangle, \langle 82, 75 \rangle \}$

画出其逻辑结构表示，指出是什么类型？

逻辑结构的类型

【例1.3】有一种数据结构 $B_2 = (D, R)$ ，其中

$D = \{48, 25, 64, 57, 82, 36, 75\}$

$R = \{r_1, r_2\}$

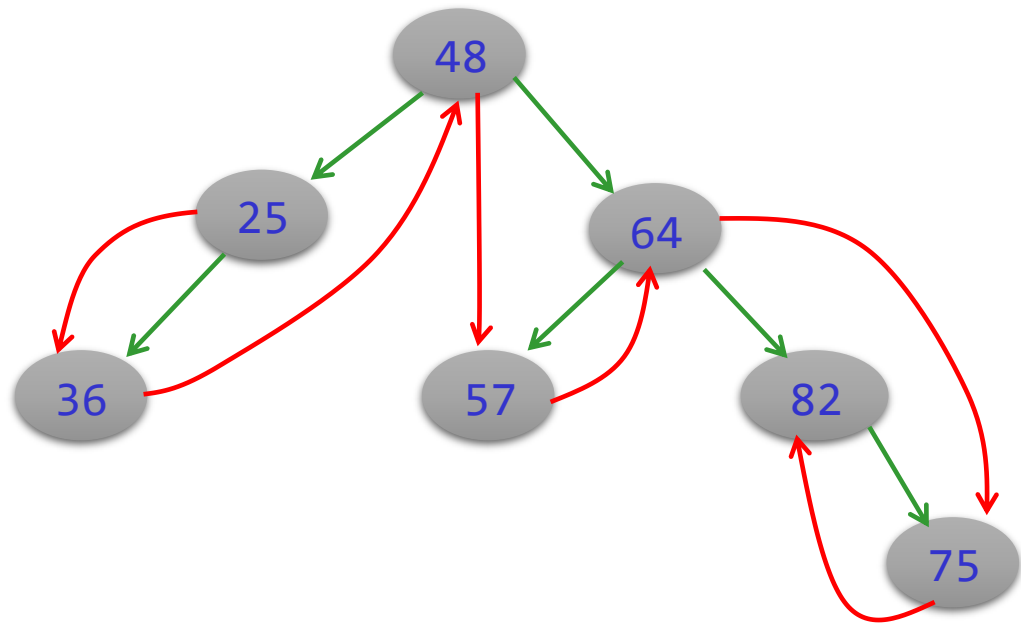
$r_1 = \{ \langle 25, 36 \rangle, \langle 36, 48 \rangle, \langle 48, 57 \rangle, \langle 57, 64 \rangle, \langle 64, 75 \rangle, \langle 75, 82 \rangle \}$

$r_2 = \{ \langle 48, 25 \rangle, \langle 48, 64 \rangle, \langle 64, 57 \rangle, \langle 64, 82 \rangle, \langle 25, 36 \rangle, \langle 82, 75 \rangle \}$

B_2 的逻辑结构图如下。

r_1 为线性结构

r_2 为树形结构



存储结构分为：

顺序存储结构——借助元素在存储器中的**相对位置**来表示数据元素间的逻辑关系

c语言中用数组来实现顺序存储结构

链式存储结构——借助指示元素存储地址的**指针**表示数据元素间的逻辑关系

c语言中用指针来实现链式存储结构

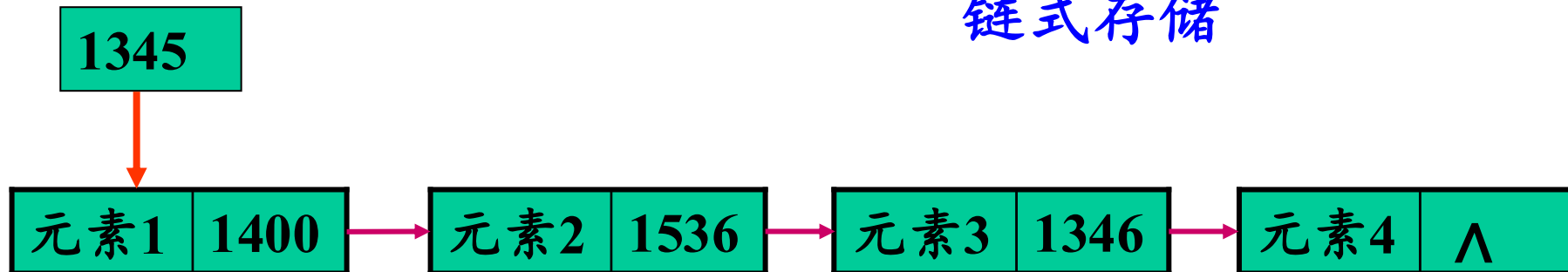
顺序存储

存储地址	存储内容
L_0	元素1
L_0+m	元素2
	
$L_0+(i-1)*m$	元素i
	
$L_0+(n-1)*m$	元素n

$$\text{Loc(元素i)} = L_0 + (i-1)*m$$

头指针 h

链式存储

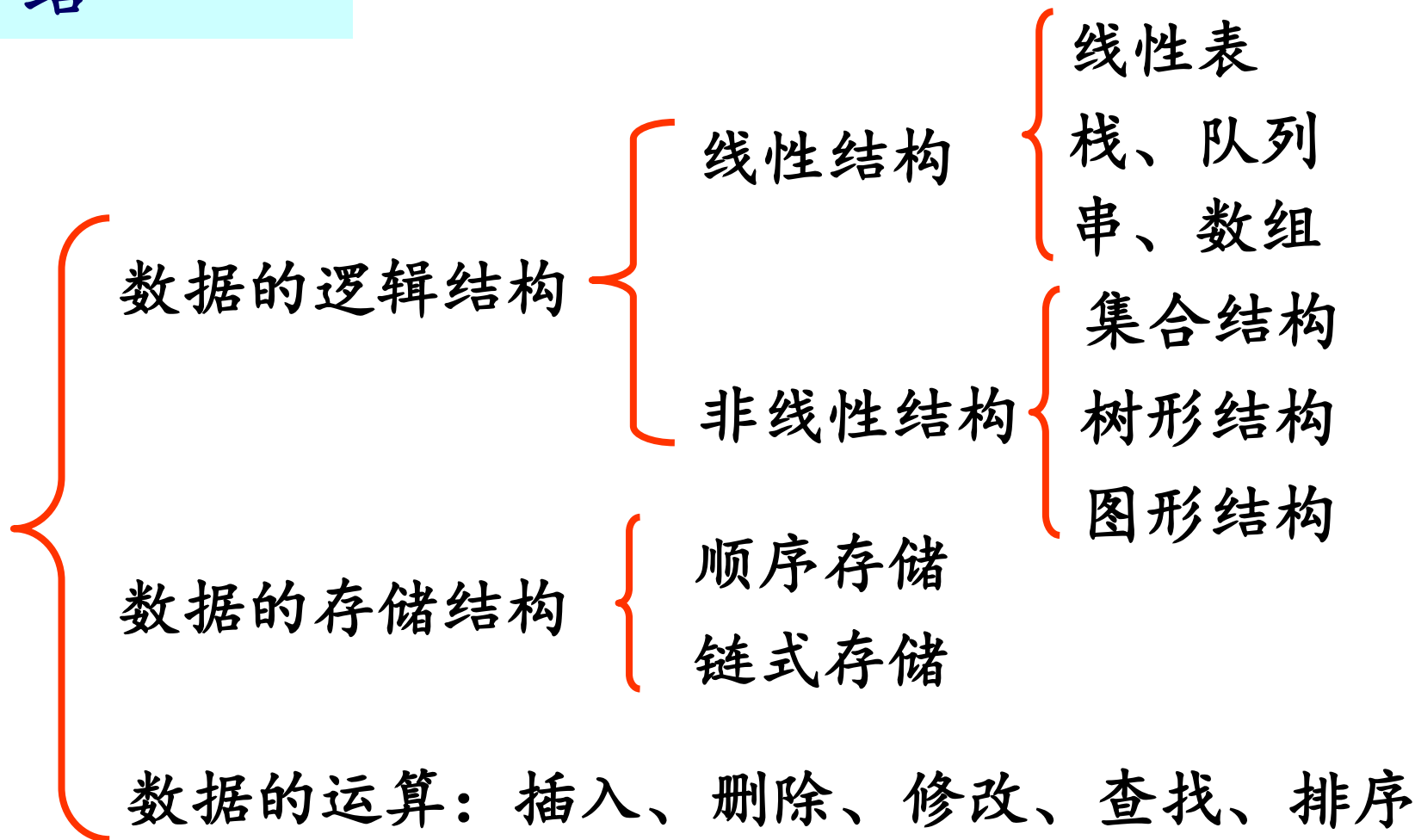


存储地址	存储内容	指针
1345	元素1	1400
1346	元素4	Λ
.....		
1400	元素2	1536
.....		
1536	元素3	1346

数据的运算

- 逻辑结构和存储结构都相同,但运算不同,则数据结构不同. 例如,栈与队列
- 对于一种数据结构,常见的运算
 - 插入
 - 删除
 - 修改
 - 查找
 - 排序

小结



运算的实现

依赖于

存储结构



存储结构

不唯一

逻辑结构

唯一

1.3 抽象数据类型的表示与实现



数据类型

- **定义：**在一种程序设计语言中，变量所具有的数据种类

C语言：

基本数据类型： char int float double void

构造数据类型：数组、结构体、指针等

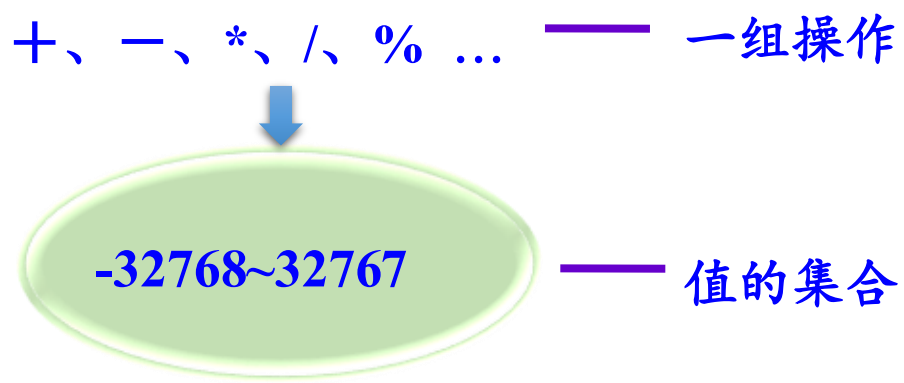
自定义数据类型： typedef

- ✓ 数据类型是一组性质相同的值的集合和定义在此集合上的一组操作的总称

数据类型

- ✓ 高级语言中的数据类型明显地或隐含地规定了在程序执行期间变量和表达的所有可能的取值范围、以及允许进行的操作。

例如，C/C++中的short int就是整型数据类型。



数据类型

例如，int数据类型：

int $i=2$, $j=5$, k ;

$k=i+j$;

...



✓ 因为 i 、 j 和 k 都属于int，而int提供了各种运算，所以可以进行相应运算。

short int $i=9999999999$;

i^{**} ;



✗ 数据取值超界

✗ 数据类型不存在该运算符

数据类型和数据结构的关系：数据类型就是已经实现了的数据结构。

抽象数据类型

(ADTs: Abstract Data Types)

指一个**数学模型**以及定义在该模型上的一组**操作**。



抽象数据类型

(ADTs: Abstract Data Types)

指一个**数学模型**以及定义在该模型上的一组**操作**。

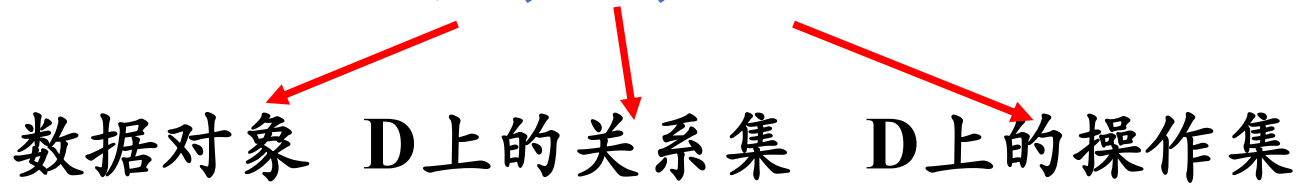
- ◆ 更高层次的数据抽象，不考虑计算机内的具体存储结构与运算的具体实现方法。
- ◆ 由用户定义，用以表示应用问题的**数据模型**
- ◆ 由**基本的数据类型**组成, 并包括**一组相关的**操作
- ◆ **抽象性**: 强调数据类型的数学特性, 与其在计算机内部如何表示和实现无关。
- ◆ **可扩展性**: 由已有的数据类型可扩展出尚未实现的数据类型。

抽象数据类型的形式定义

抽象数据类型可以用以下的三元组来表示：

$$\text{ADT} = (\mathbf{D}, \mathbf{S}, \mathbf{P})$$

数据对象 $\mathbf{D}$ 上的关系集 $\mathbf{D}$ 上的操作集



抽象数据类型的定义格式

ADT 常用 定义 格式

ADT 抽象数据类型名{

数据对象：<数据对象的定义>

数据关系：<数据关系的定义>

基本操作：<基本操作的定义>

} **ADT** 抽象数据类型名

抽象数据类型 = 逻辑结构 + 抽象运算

抽象数据类型的定义格式

ADT 常用 定义 格式

ADT 抽象数据类型名{

数据对象：<数据对象的定义>

数据关系：<数据关系的定义>

基本操作：<基本操作的定义>

} **ADT** 抽象数据类型名

其中基本操作的定义格式为：

- 基本操作名（参数表）
- 初始条件：<初始条件描述>
- 操作结果：<操作结果表示>

抽象数据类型的定义格式

例如，定义复数抽象数据类型Complex

ADT Complex {

一个复数的形式： $e_1 + e_2i$ 或 (e_1, e_2)

数据对象：

$D = \{ e_1, e_2 \mid e_1, e_2 \text{ 均为实数} \}$

数据关系：

$R = \{ \langle e_1, e_2 \rangle \mid e_1 \text{ 是复数的实部, } e_2 \text{ 是复数的虚部} \}$

基本运算：

AssignComplex(&z, v1, v2): 构造复数Z。

DestroyComplex(&z): 复数z被销毁。

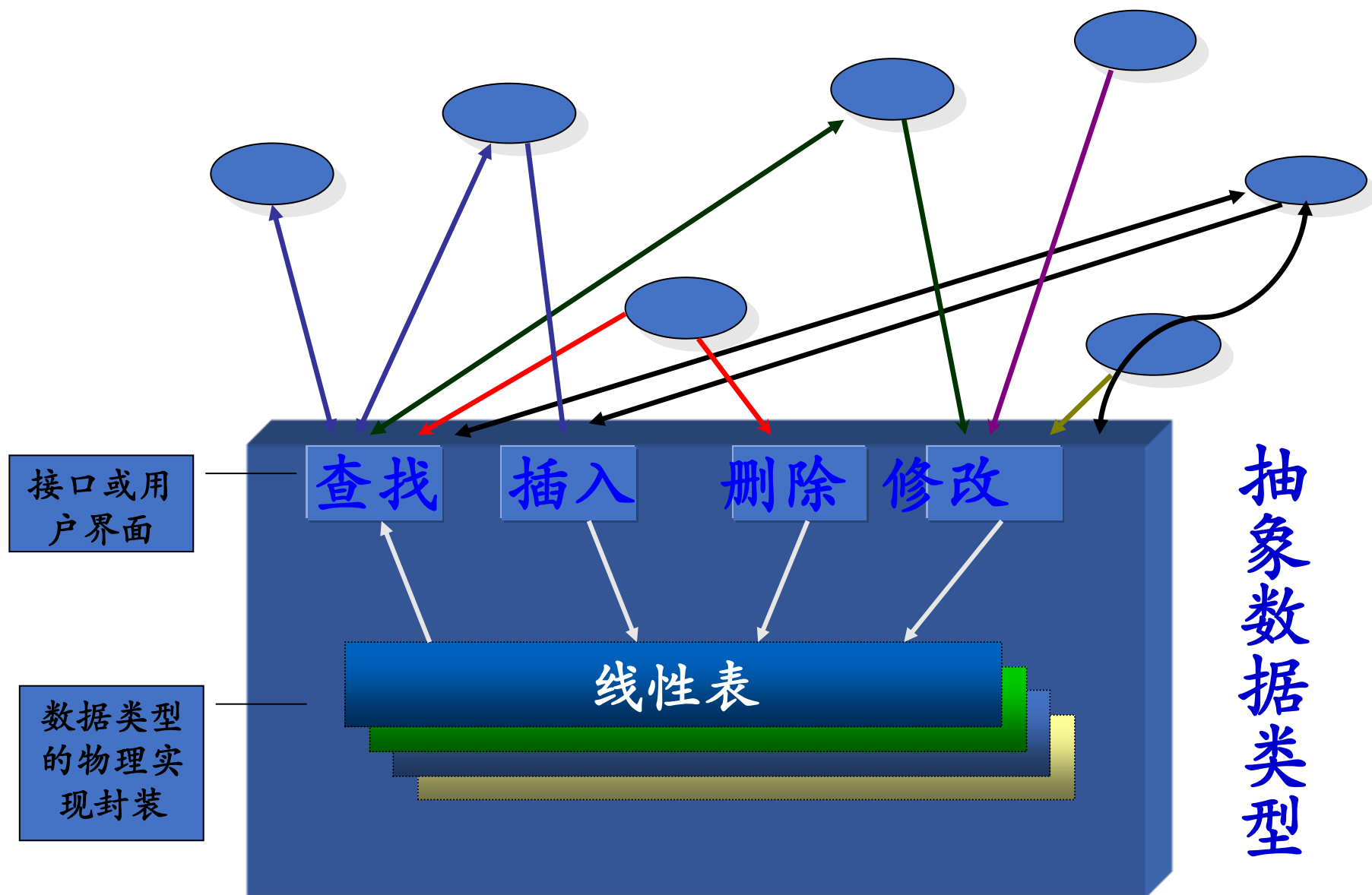
GetReal(z, &real): 返回复数z的实部值。

GetImag(z, &Imag): 返回复数z的虚部值。

Add(z1, z2, &sum): 返回两个复数z1、z2的和。

} ADT Complex

信息隐蔽和数据封装，使用与实现相分离



抽象数据类型的实现

注意：一个问题抽象为一个抽象数据类型后，仅仅是形式上的抽象定义，还没有达到问题解决的目的，要实现这个目标，就把抽象数据类型的定义转化为可以在计算机上实现的某种语言的代码，变为一个能用的具体的数据类型。

C语言实现抽象数据类型

- 用已有的数据类型定义描述它的存储结构
- 用函数定义描述它的操作



使用抽象数据类型

抽象数据类型的实现

抽象数据类型可以通过**固有的**数据类型（如整型、实型、字符型等）来表示和实现。

它有些类似C语言中的**结构体 (struct) 类型**，但增加了相关的**操作**。

大部分教材中用的是**类C语言**（介于**伪码**和C语言之间）作为描述工具。

```
Begin (算法开始)
  输入 A, B, C
  IF A>B 则
    A→Max
  否则    B→Max
  IF C>Max 则    C→Max
Print  Max
End (算法结束)
```

但上机时要用具体语言实现，如C或C++等

抽象数据类型的实现

(1) 预定义常量及类型

- //函数结果状态代码

```
#define OK 1
```

```
#define ERROR 0
```

```
#define OVERFLOW -2
```

- // Status是函数返回值类型，其值是函数结果状态代码。

```
typedef int Status;
```

(2)数据元素被约定为ElemType 类型，用户需要根据具体情况，自行定义该数据类型。

(3)算法描述为以下的函数形式：

函数类型 函数名（函数参数表）

{

语句序列；

}

(4) 内存的动态分配与释放

使用new和delete动态分配和释放内存空间

分配空间 指针变量=new数据类型;

释放空间 delete指针变量;

(5) 赋值语句

(6) 选择语句

(7) 循环语句

(8) 使用的结束语句形式有:

函数结束语句 `return`

循环结束语句 `break;`

异常结束语句 `exit (异常代码) ;`

(9) 输入输出语句形式有:

输入语句 `cin>>变量1;`

输出语句 `cout<<表达式1;`

(10) 扩展函数有:

求最大值 `Max`

求最小值 `Min`

抽象数据类型的实现

例如，抽象数据类型“复数”的c语言实现

```
typedef struct {  
    float real; //实部  
    float imag; //虚部  
} Complex; //使用已存在的struct类型定义复数抽象类型
```

//函数声明

```
void assign (Complex*A, float real, float imag); //赋值  
void add (Complex*C, Complex A, Complex B); //相加
```

....

抽象数据类型的实现

例如，抽象数据类型“复数”的c语言实现

```
void assign (Complex*A, float real, float imag) {  
    A->real = real; //实部赋值  
    A->imag = imag; //虚部赋值  
} //构造复数，赋值给A  
  
void add (Complex*C, Complex A, Complex B) {  
    C->real = A.real + B.real; //实部相加  
    C->imag = B.imag + A.imag; //虚部相加  
}
```

注意：Complex是我们定义的一个结构体类型
带*：指针变量，是指向Complex类型的指针；
不带*：Complex类型的普通变量

抽象数据类型的实现

进一步，使用定义的抽象数据类型“复数”解决问题。

求和： $z_1 = 5 + 6i$, $z_2 = 12 + 72i$

```
# include <studio.h>

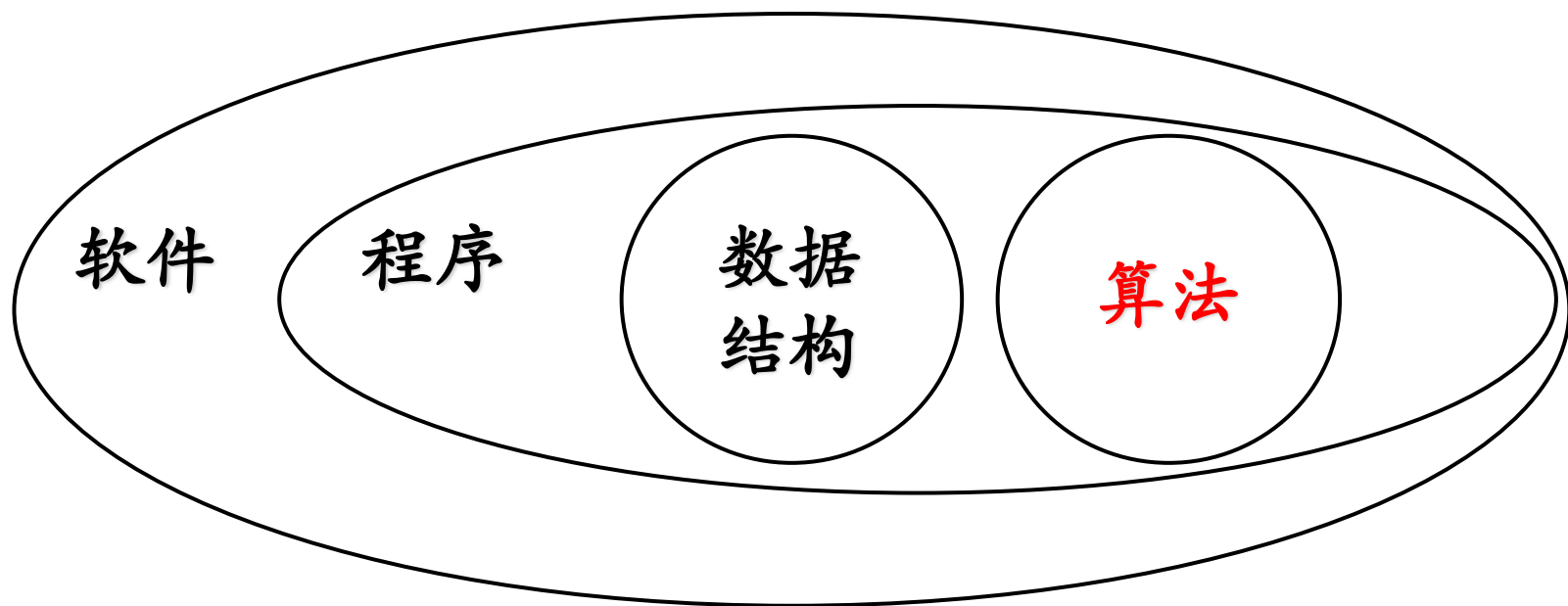
... ..

void main ()
{
    Complex z1, z2, z3;
    float real, imag;
    assign (z1, 5, 6); //构造复数z1
    assign (z2, 12, 72); //构造复数z2
    add (z3, z1, z2); //复数求和
}
```



1.4 算法和算法分析

算法的定义



- 算法定义：一个有穷的指令集，这些指令为解决某一特定任务规定了一个运算序列

- ◆ 自然语言：简单便于理解、不够严谨
- ◆ 流程图：简洁明了、便于理解交流
- ◆ 程序设计语言：求高
- ◆ 伪码 (Pseudo code)：更容易描述和被算法框图更接近

Algorithm 1: Framework of the Proposed SparseEA

Input: N (population size)
Output: P (final population)

```
1  $[P, Score] \leftarrow Initialization(N)$ ; //Algorithm 2
2  $[F_1, F_2, \dots] \leftarrow$  Do non-dominated sorting on  $P$ ;
3  $CrowdDis \leftarrow CrowdingDistance(F)$ ;
4 while termination criterion not fulfilled do
5    $P' \leftarrow$  Select  $2N$  parents via binary tournament selection
     according to the nondominated front number and  $CrowdDis$ 
     of solutions in  $P$ ;
6    $P \leftarrow P \cup Variation(P', Score)$ ; //Algorithm 3
7   Delete duplicated solutions from  $P$ ;
8    $[F_1, F_2, \dots] \leftarrow$  Do non-dominated sorting on  $P$ ; //  $F_i$ 
     denotes the solution set in the  $i$ -th
     non-dominated front
9    $CrowdDis \leftarrow CrowdingDistance(F)$ ;
10   $k \leftarrow \operatorname{argmin}_i |F_1 \cup \dots \cup F_i| \geq N$ ;
11  Delete  $|F_1 \cup \dots \cup F_k| - N$  solutions from  $F_k$  with the
     smallest  $CrowdDis$ ;
12   $P \leftarrow F_1 \cup \dots \cup F_k$ ;
13 return  $P$ ;
```

算法的特性

- ◆ **有输入** 有0个或多个输入
- ◆ **有输出** 有一个或多个输出(处理结果)
- ◆ **确定性** 每步定义都是确切、无歧义的
- ◆ **有穷性** 算法应在执行有穷步后结束
- ◆ **可行性** 可以通过已实现的基本操作执行

算法的特性

【例】考虑下列两段描述，这两段描述均不能满足算法的特性，试问它们违反了哪些特性？

(1) 描述一

```
void exam1()
{
    int n=2;
    while (n%2==0)
        n=n+2;
    printf("%d\n", n);
}
```

其中有一个死循环，违反了算法的有穷性特性。

算法的特性

(2) 描述二

```
void exam2()
{ int x, y;
  y=0;
  x=5/y;
  printf("%d, %d\n", x, y);
}
```

其中包含除零错误，违反了
算法的可行性特性

算法的评价

通常设计一个“好”的算法应达到以下目标：

- ◆ 正确性
- ◆ 可读性
- ◆ 健壮性

当输入的数据非法时，算法应恰当作出反应或进行相应处理，而不是产生莫名其妙的输出结果。不应中断执行，二是返回一个表示错误或错误性质的值，以便在更高层次上处理。

- ◆ 高效性（时间代价和空间代价）

时间复杂度和空间复杂度是衡量算法的两个主要指标。两者都与问题规模有关。